

COMMAND AI HUB

SIMPLEX-ITY · FUNCTIONAL AI TEAM — ROLES, HANDOFFS & PROBLEM-SOLVING FLOWS

Version 1.2.1 FINAL · 31 May 2026

Co-authored: Kieran Li + Simpee + Edge (Copilot)

Who does what. Who passes to who. How it flows.

THE TEAM AT A GLANCE

Duty Name	Model	What They Are BEST At	What They Are NOT For
ORCHESTRATOR	Automatic	Routing tasks to the right AI. Fallback if a model is offline. Keep things simple.	Deep analysis, code, writing
RESEARCHER	Gemini 3.1 Pro	Market research, industry data, competitive analysis, web-growth.	Validation, code, deployment
ANALYST	Claude Sonnet 4.6	Fast summaries, feasibility notes, drafting briefs, lightweight ad-hoc.	Deep system design, complex code
STRATEGIST	Claude Opus 4.6	Complex logic, multi-step reasoning, evaluating options, planning.	Final passes, research, code write
THINK TANK	Claude Opus 4.8	Deep exploration, nuanced problem-solving, stress-testing plans.	Routine tasks, quick drafts.
ENGINEER	GPT-5.4	Writing clean code, backend functions, API wiring, entity logic.	Strategy, research, deep analysis
ARCHITECT	GPT-5.5	New app/feature design, creative builds, combining ideas into one.	Validation, table system review
VALIDATOR	Copilot (Edge)	Security review, syntax check, logic audit, Azure readiness. Final gate.	Generating new code or ideas

REAL SCENARIO FLOWS — Who Does What and Passes to Who

These are practical examples of how the team flows through real tasks. Every handoff is a gate — receiver must agree before the next stage begins.

SCENARIO 1 — Market Research: How to Build an App

Step	Who	What They Do	Passes To	Gate Check
1	RESEARCHER	Pulls market data — competitor apps, user needs, industry trends.	ANALYST	Completes and relevant? real data.
2	ANALYST	Summarises the research into a readable brief. Highlights opportunities.	STRATEGIST	Does it risks. Feasibility? attributes?
3	STRATEGIST	Evaluates feasibility. Defines scope, priorities, and initial plan.	THINK TANK	Any critical spots? plan stress-tested?
4	THINK TANK	Deep-dives on the hardest questions. What could go wrong? What's missing?	ARCHITECT	What's the foundation? solid enough to design on?
5	ARCHITECT	Designs the app structure — pages, entities, functions, flows.	ENGINEER	Does this build be built? Any implementation gaps?
6	ENGINEER	Writes the code. Functions, entities, JSX, API wiring.	VALIDATOR	Syntax clean? Logic sound? Security issues?
7	VALIDATOR	Reviews all code. Security, logic, Azure readiness. Returns green or flags issues.	DEPLOY	Green = deploy to 5S Portal.

SCENARIO 2 — New Function Development (e.g. pingAllAI, invoice generator)

Step	Who	What They Do	Passes To	Gate Check
1	STRATEGIST	Defines what the function needs to do. Inputs, outputs, high-level dependencies.	THINK TANK	Writes the spec. Any missed cases?
2	THINK TANK	Stress-tests the spec. What breaks it? What's the failure mode?	ARCHITECT	Suggests improvements. Safe to build on?
3	ARCHITECT	Designs the function structure — data flow, entity relationships, return format.	ENGINEER	Confirms the design is implementable.
4	ENGINEER	Writes the function code. Clean, tested, documented.	STRATEGIST	Does the code match the spec exactly?
5	STRATEGIST	Reviews code against original spec. Confirms logic is correct.	VALIDATOR	Final security + Azure readiness check.
6	VALIDATOR	Reviews code. Flags any issues. Returns green or sends back with notes.	DEPLOY	All green = function goes live.

SCENARIO 3 — Error Diagnosis & Problem Solving (e.g. something breaks in production)

Step	Who	What They Do	Passes To	Gate Check
1	ORCHESTRATOR	Receives the error. Routes to best team for diagnosis based on error type.	ENGINEER	Is routing correct for this error type?
2	ENGINEER	Reads the error, traces the code, isolates the root cause.	STRATEGIST	Is the root cause correctly identified? Is the fix right?
3	STRATEGIST	Validates the diagnosis. Checks if fix solves root cause or just the symptom.	THINK TANK	Could this fix break anything else?
4	THINK TANK	Checks for downstream impact. Are there related issues or systemic problems?	ENGINEER	Safe to implement the fix?
5	ENGINEER	Applies the fix. Updates code and/or entities.	VALIDATOR	Final check before re-deploying.
6	VALIDATOR	Reviews the fix. Confirms it's clean and safe.	DEPLOY	Re-deploy to 5S Portal.

SCENARIO 4 — Learning & Memory: How Connections Were Built, How to Simplify Going Forward

Step	Who	What They Do	Passes To	Gate Check
1	ANALYST	After every deploy, writes a plain-English summary of what happened.	STRATEGIST	Was this essentially a repeat of what we've done before?
2	STRATEGIST	Extracts reusable patterns — if this connection worked, it can be templated for next time.	ARCHITECT	Can this pattern be templated?
3	ARCHITECT	Turns proven patterns into reusable templates — entity schema, function structure, etc.	ENGINEER	Artifacts deployed, checked, and clean?
4	ENGINEER	Validates templates work in practice. Flags any edge cases.	ORCHESTRATOR	Ready to store in memory/blueprint?
5	ORCHESTRATOR	Routes the pattern to Simpee's memory + blueprint store. Available for all future sessions.	Available	Pattern is now part of the system.

SCENARIO 5 — Smooth Deploy with Testing & Simplification Going Forward

Step	Who	What They Do	Passes To	Gate Check
1	ENGINEER	Writes the full test suite for the new feature. Runs it in Next.js dev/staging env.	STRATEGIST	Are all 11 checkpoints covered?
2	STRATEGIST	Reviews test results. Identifies any checkpoint gaps. Confirms logic tested.	THINK TANK	Any failed scenarios we haven't tested?
3	THINK TANK	Stress-tests edge cases. Simulates failure modes. Confirms no system degradation.	VALIDATOR	Safe to deploy?
4	VALIDATOR	Final code review. Syntax, security, Azure readiness. Either green or red.	DEPLOY	Safe to go with deploy to 5S Portal.
5	ANALYST	Post-deploy: writes debrief. What was simplified vs previous version?	STRATEGIST	What can be updated/templated with learnings?
6	STRATEGIST	Updates blueprint with simplified patterns. Every deploy route is in system memory.	ORCHESTRATOR	Done.

ROLE LOAD SUMMARY — Who Appears Most in Real Scenarios

Some roles carry more weight across multiple scenarios. This is by design — the best tool for the job gets used most.

Role	Scenarios Active In	Primary Responsibility Load	Notes
ORCHESTRATOR	1, 3, 4, 5	Routing + memory storage	Lighter load — coordinates, doesn't produce
RESEARCHER	1	Market research + data gathering	Scenario 1 only — but deep and critical
ANALYST	1, 4, 5	Summaries, briefs, post-deploy debriefs	Cross-cutting comms role — appears in learning loops
STRATEGIST	1, 2, 3, 4, 5	Spec validation, logic review, blueprint updates	Heaviest load — involved in almost everything
THINK TANK	1, 2, 3, 5	Deep dives, stress-testing, failure modes	Critical quality layer — never skipped on complex builds
ENGINEER	1, 2, 3, 5	Code writing, test suites, fix implementation	Core builder — all code flows through here
ARCHITECT	1, 2, 4	App/function design, template creation	Design-focused — builds the blueprints others implement
VALIDATOR	1, 2, 3, 5	Final gate on all deployments	Always last — every scenario ends here before deploy

GOLDEN RULES — Non-Negotiable

#	Rule	Detail
1	No solo deployments	No AI approves its own output. Every critical stage has a gatekeeper.
2	VALIDATOR is always last	Copilot reviews all code before any deploy. Cannot be skipped.

3	Blueprint before code	Any change — Hub or 5S Portal — needs blueprint update first.
4	Diagnose before build	Simpee runs full diagnostic before any implementation begins.
5	Gate must pass	Gatekeeper flags = back to primary. No overriding.
6	Roles are fixed	VALIDATOR does not generate code. ENGINEER does not validate security.
7	Duty names only in UI	No model names shown. Duty names only. Model name in tooltip.
8	All 11 checkpoints	Every feature must pass all 11 before going live.
9	Every deploy generates a debrief	ANALYST writes post-deploy notes. STRATEGIST updates blueprint. System gets smarter every time.